

EC-341 Digital System Design

BS(CE)-2021

LAB REPORT # 05



Submitted By:

Reg. No.	Name
21-CE-009	Huzaifa Shahbaz
21-CE-021	Yawar Ali Malik
21-CE-020	Muhammad Ibthesam Khan

Department of Computer Engineering

BS Computer Engineering Program

HITEC University Taxila

	Presentation (Format(0.5)+ Title (0.5)+ Conclusion(1) +Procedure(2) +Objective(1))	Calculation/ Coding	Observation/ Program Results	Total
Total	5	5	5	15
Obtained				

Lab Instructor.
Engr. Fasih Ahmad

Experiment # 05

Implementation of Combinational Circuits

Objective

- Combinational circuits
- The Adder
- Lab tasks

Software Used

- Vivado

Procedure

We begin by creating Verilog modules using behavioral modeling for the half adder, full adder, 2-bit adder, and 2-bit 2's complement adder. Define inputs and outputs for each module and connect switches for inputs and LEDs for outputs on the FPGA board. Extend the Verilog modules as necessary, incorporating additional logic for carry inputs and overflow conditions. Synthesize the designs and program the FPGA, ensuring proper pin assignments. Test the functionality of each circuit by toggling input switches and observing the corresponding LED outputs, verifying correct computation of sums and carries, as well as handling of negative numbers and overflow conditions in the 2's complement adder. Document the results and any observations in the report for clarity and reference.

Tasks

Constraint file:

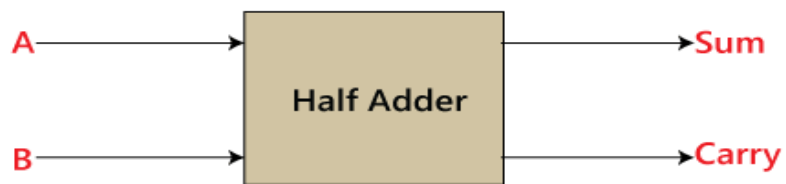
NOTE: This is used the same in all codes below,

```
###Switches
set_property -dict { PACKAGE_PIN G15   IOSTANDARD LVCMOS33 } [get_ports { switch1 }]; #IO_L19N_T3_VREF_35 Sch=SW0
set_property -dict { PACKAGE_PIN P15   IOSTANDARD LVCMOS33 } [get_ports { switch2 }]; #IO_L24P_T3_34 Sch=SW1
set_property -dict { PACKAGE_PIN W13   IOSTANDARD LVCMOS33 } [get_ports { switch3 }]; #IO_L4N_T0_34 Sch=SW2
set_property -dict { PACKAGE_PIN T16   IOSTANDARD LVCMOS33 } [get_ports { switch4 }]; #IO_L9P_T1_DQS_34 Sch=SW3

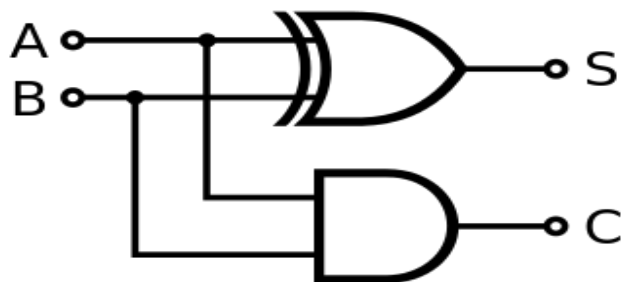
26 ###LEDs
27 set_property -dict { PACKAGE_PIN M14   IOSTANDARD LVCMOS33 } [get_ports { LED1 }]; #IO_L23P_T3_35 Sch=LED0
28 set_property -dict { PACKAGE_PIN M15   IOSTANDARD LVCMOS33 } [get_ports { LED2 }]; #IO_L23N_T3_35 Sch=LED1
29 set_property -dict { PACKAGE_PIN G14   IOSTANDARD LVCMOS33 } [get_ports { LED3 }]; #IO_0_35=Sch=LED2
30 set_property -dict { PACKAGE_PIN D18   IOSTANDARD LVCMOS33 } [get_ports { LED4 }]; #IO_L3N_T0_DQS_AD1N_35 Sch=LED3
31
```

Task 1:

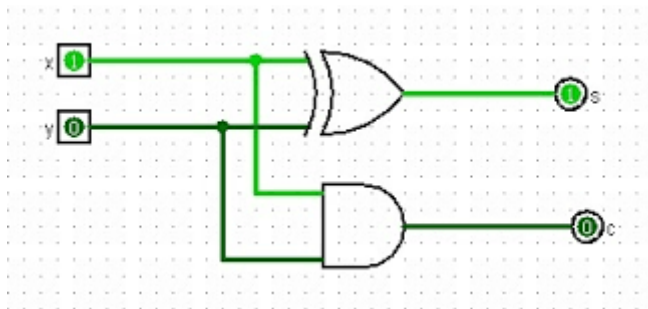
Block Diagram:



Circuit:



Logism Implementation:



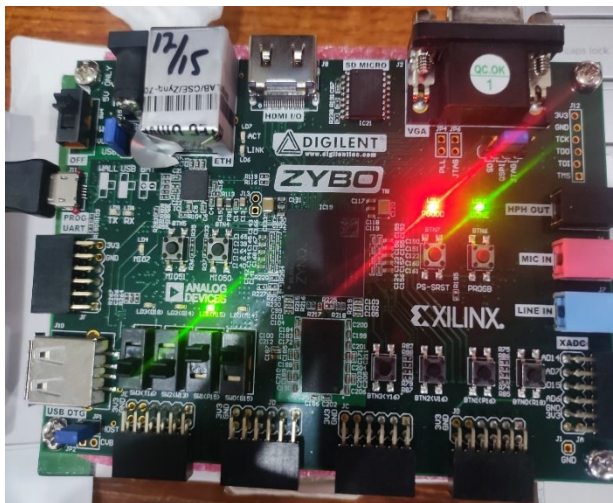
Truth Table:

Truth Table			
Input		Output	
A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Code:

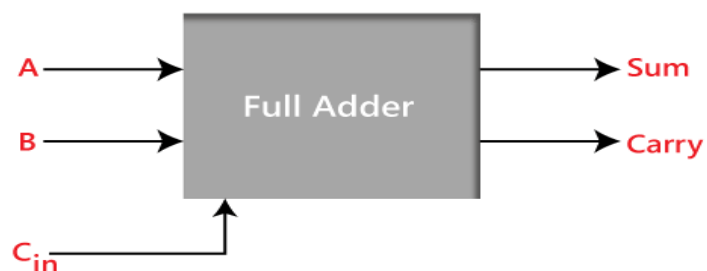
```
module Hadd(input switch1,input switch2,output reg LED1,output reg LED2);  
    always@(switch1 or switch2)  
    begin  
        LED1=switch1^switch2;  
        LED2=switch1&switch2;  
    end  
endmodule
```

Output:

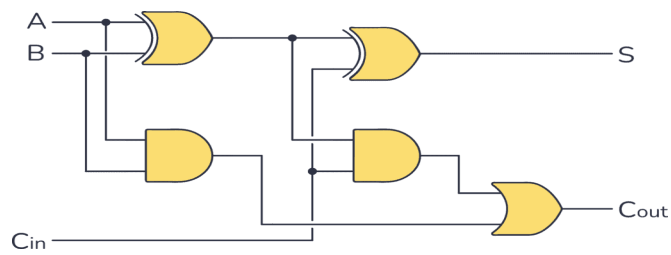


Task 2

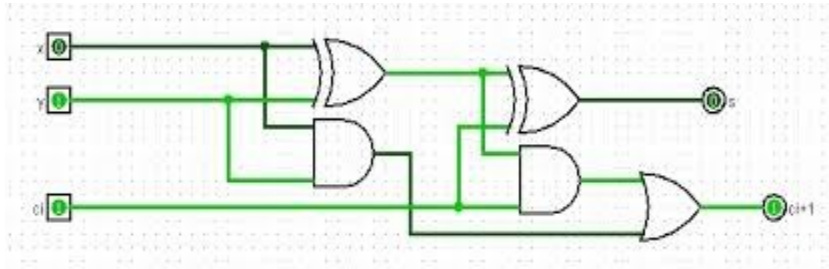
Block Diagram:



Circuit:



Logism Implementation:



Truth Table:

Input			Output	
A	B	Cin	Sum	Carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Code:

```
module Fudd(  
    input switch1,  
    input switch2,  
    input switch3,  
    output reg LED1,  
    output reg LED2  
);  
always@(switch1 or switch2 or switch3)  
begin
```

```
LED1=switch1^switch2^switch3;
```

```
LED2=switch3&(switch1^switch2)|(switch1&switch2);
```

```
end
```

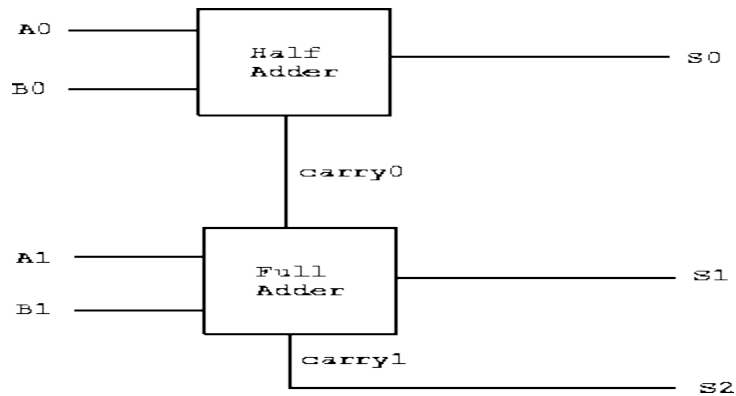
```
endmodule
```

Output:

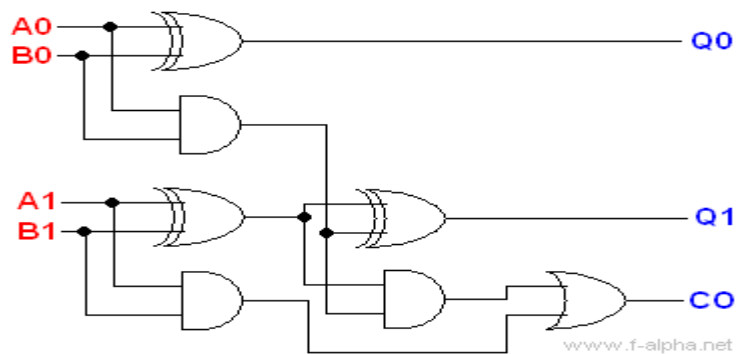


Task 3

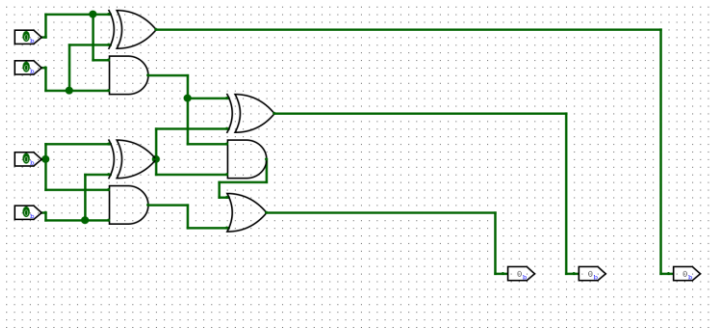
Block Diagram:



Circuit:



Logism Implementation:



Truth Table:

INPUTS					OUTPUTS		
A0	A1	B0	B1	Ci	S0	S1	Co
0	0	0	0	0	0	0	0
1	0	0	0	0	1	0	0
0	1	0	0	0	0	1	0
1	0	1	0	0	0	1	0
0	1	0	1	0	0	0	1
0	1	0	0	1	1	1	0
1	0	1	0	1	1	1	0
1	1	1	1	1	1	1	1

Code:

```
module tadd(  
    input switch1,  
    input switch2,  
    input switch3,  
    input switch4,  
    output reg LED1,  
    output reg LED2,  
    output reg LED3,  
    output reg LED4  
);  
always@(switch1 or switch2 or switch3 or switch4)  
begin  
    LED1=switch1^switch2;
```

```

LED2=switch1&switch2;
LED3=switch3^switch4^LED2;
LED4=LED2&(switch3^switch4)|(switch3&switch4);
end
endmodule

```

Output:



Task 4

```

module code (
    input [1:0] switch1,
    output reg [1:0] LED1
);
    always @(*) begin
        if (switch1 == 2'b11)
            LED1 = 2'b00;
        else
            LED1 = (~switch1) + 2'b01;
        end
    endmodule

```


Output:



Conclusion

In this lab, we successfully implemented various combinational circuits, including the half adder, full adder, 2-bit adder, and 2-bit 2's complement adder. Utilizing switches and LEDs as input and output interfaces respectively, we demonstrated the functionality of these circuits on an FPGA platform. Through this hands-on exercise, we gained valuable insights into digital system design.